

Continuous Colony Health Monitoring for Apiaries Using a Fog-to-Cloud Architecture

Yashaswini Penumarthy
MSc in Cloud Computing
Fog and Edge Computing (H9FECC)
National College of Ireland
Student ID: X24262404
x24262404@student.ncirl.ie

Abstract—A honeybee colony can slip from healthy into starvation, chilled brood, or the run-up to a swarm within days, yet an apiary is often inspected only every week or two and is frequently sited far from the beekeeper. This paper presents a working fog-to-cloud pipeline that watches two apiaries continuously and, rather than only report raw numbers, reasons about colony state. Each hive carries five instruments, weight, brood-nest temperature, internal humidity, acoustic buzz frequency, and entrance traffic, that are sampled locally and dispatched to a fog gateway at the apiary. At the gateway each stream is gathered into a fixed window, distilled to summary statistics, and run against colony rules that flag brood overheating, chilling, starvation, and the acoustic signature preceding a swarm. Every window’s small aggregate then rides a managed message queue to an event-driven ingestion function that stores it, and a separate, independently deployed function answers a dashboard interface that turns those stored windows into a plain-language colony-health summary per apiary, with the operator page served as static content from object storage. The system is built in Node.js and provisioned to a real Amazon Web Services account by a single infrastructure-as-code apply that created twenty-four resources. Measured at the live endpoint, the pipeline comes up healthy end to end inside about a minute and the stored record count grows from poll to poll, with both apiaries rendering live readings, per-hive narratives, and firing brood-overheat alerts in a browser.

Index Terms—fog computing, edge computing, precision apiculture, colony health monitoring, serverless, message queue, Internet of Things.

I. INTRODUCTION

Beekeepers have long faced high and variable colony losses, and a colony’s decline is often well advanced before a routine inspection reveals it. Opening a hive is itself disruptive, it chills the brood nest and provokes the bees, so inspection is deliberately infrequent, which leaves long blind intervals during which a queen can fail, stores can run out, or a colony can prepare to swarm. Precision apiculture responds to this by instrumenting the hive so that its state can be observed without opening it, and a body of work has shown that continuously logged signals such as weight, internal temperature, and sound carry rich information about colony condition [1], [2]. An apiary, moreover, is often remote, at the edge of a field or on a rooftop, so the sensing problem is naturally a wireless-sensor-network problem, with the attendant constraints on bandwidth and connectivity [3].

Those constraints make a naive design, streaming every raw reading to a central cloud, a poor fit. A hive scale, a temperature probe, and a microphone together produce readings far faster than any single reading is useful, the uplink from a remote apiary is limited and intermittent, and a monitoring view that fails whenever connectivity drops is of least use in the exposed locations apiaries occupy. Edge and fog computing meet these constraints by drawing computation nearer the source of the data [4]. Setting a fog tier between the hives and the cloud keeps the windowing, the statistical reduction, and the urgent rule checks at the apiary, while the cloud is left with what suits it, durable storage, elastic querying, and an interface reachable from anywhere [5], [6]. The cloud services used here sit within the usual on-demand, elastic, metered definition of cloud computing [7].

A further observation shapes this work. Even continuous numbers are hard for a beekeeper to act on if they arrive only as five simultaneous graphs per hive; what a keeper wants to know is whether a colony is thriving, stressed, or about to swarm. This system therefore does two things beyond plain telemetry. At the edge it evaluates colony rules, including an acoustic rule, because a colony preparing to swarm changes its sound before it changes its weight. At the read tier it condenses the recent windows for each apiary into a short plain-language health summary, so the dashboard reports a judgement, not just a chart.

This paper reports the design, implementation, and live deployment of such a pipeline. The deployment watches two apiaries, each carrying five instruments. The objectives are fourfold. First, to reduce raw readings to compact per-window aggregates at a fog tier at the apiary. Second, to evaluate colony-health rules at the edge, including the acoustic precursor of swarming, so that a developing problem is flagged in the cycle it appears. Third, to build a scalable cloud back end that ingests aggregates reliably, stores them durably, and serves both the readings and a synthesised colony-health narrative to an operator dashboard without the write path and the read path contending. Fourth, to deploy the whole system to a real cloud account and verify it end to end. What follows sets out the architecture and its design decisions, the implementation together with its tests, the deployment, an evaluation drawn from the running system, and a closing reflection.

II. SYSTEM ARCHITECTURE AND DESIGN

Conceptually the design is one flow of data with a side path for the operator. Sensors at the hives feed readings to an apiary-side fog gateway, which buckets them into windows, boils them down, and tests them against colony rules; the summaries then ride a queue into the cloud, where one function stores them and a second reads them back, building a colony-health narrative as it does so. Fig. 1 lays out the deployment. The tiers are described in order, and the section closes by weighing the alternatives.

A. Instrumented Hives and the Fog Gateway

Five sensor processes stand in for each apiary, one for every measured quantity: hive weight in kilograms, brood-nest temperature in degrees Celsius, internal humidity as a percentage, acoustic buzz frequency in hertz, and entrance traffic as a bee count. To keep a signal believable, a sensor takes a bounded random walk, moving each reading a little from the one before and holding it within a physically sensible band, so the series meanders instead of leaping about. Two settings, independent of each other, drive it: one fixes how frequently a reading is taken, the other how frequently the gathered readings are sent on to the gateway. Keeping those settings apart means a quantity that changes quickly, sound above all, can be read often and yet still reach the edge in thrifty batches. What travels on each send is a small JSON body listing the sensor type, the apiary, and the unit, plus the handful of timestamped readings gathered since the last one, no more than a few hundred bytes in all.

A compact Node.js service on Express, the fog gateway receives the posted batches and maintains one live accumulator for each sensor-apiary pairing. It never keeps the raw readings; instead every value, as it arrives, is folded into a small tuple of count, running sum, minimum, maximum, and most recent reading, which holds the gateway's memory constant however many readings flow through. Once per fixed interval it closes the open window, turning each accumulator into a summary that records the window bounds, the count, the minimum, the maximum, the mean, and the last reading, and it passes that summary through the colony rules before sending it onward.

B. Edge Alert Rules for Colony Health

Four rules are evaluated; internal humidity and entrance traffic are retained for display and narrative but carry no rule, because neither has a single threshold that by itself denotes a fault. Table I lists the rules. Each rule is stored as a tuple whose field names both a sensor type and the aggregate to test, for example the mean brood-nest temperature, so a single evaluator can apply every rule uniformly. Three of the four watch the window mean, because a sustained average is a more reliable signal of a genuine colony condition than one noisy sample. The swarming rule watches the mean acoustic buzz frequency: a colony preparing to swarm raises the pitch of its collective sound, so an elevated mean frequency is an early, non-invasive indicator that precedes the visible signs. Evaluating these rules at the edge means a developing

TABLE I
EDGE-EVALUATED COLONY-HEALTH RULES

Signal	Rule (per window)	Alert raised
Brood temperature	mean above 36 °C	Brood overheating
Brood temperature	mean below 32 °C	Brood chilling
Hive weight	mean below 20 kg	Colony starvation
Acoustic buzz	mean above 350 Hz	Swarming precursor
Humidity, traffic	(informational, no rule)	None

condition is flagged in the same cycle as the readings that reveal it, without a round trip to the cloud.

A closing window does not prompt one message per hive per signal. The gateway instead assembles all of that cycle's summaries and sends them together in a single batched call, dividing them into smaller groups only where the queue limits how many entries one call can hold. Since two apiaries yield at most ten summaries a cycle, what would be ten separate cloud calls becomes one, cutting both request cost and the per-message overhead.

C. Queue-Decoupled Ingestion and the Reading Store

The aggregates do not go straight into the storage function; they are placed on a managed message queue first. That queue evens out the two sides of the exchange, keeping messages safely until a consumer takes them, so a consumer that lags or drops offline briefly loses nothing and never stalls the gateway, and any sudden rush is paced down to something the consumer can handle, which is precisely the producer-consumer separation message-oriented middleware is known for [8]. That protection begins once a message is enqueued and does not cover the enqueue itself: if the gateway's batched call fails, the window is already sealed and emptied, so instead of retrying or holding the data the gateway notes the failure and continues, an accepted compromise given how little two apiaries produce.

Draining the queue is an event-driven ingestion function that recasts each aggregate as a stored record and puts it in a managed key-value store, under a key of two parts. The partition key is the sensor type; gathering every reading of one quantity in one place lets the common request, the recent windows for a given signal, resolve in a single lookup. The sort key joins the window-end timestamp to the apiary label. Ordering by timestamp keeps each partition chronological, and adding the apiary label is a deliberate touch: two apiaries closing the same signal in one cycle write records that match on both partition key and window-end time, so absent the apiary label the later write would erase the earlier. Joining the label keeps the two apiaries' records apart within one partition.

D. Read Tier: Interface, Colony-Health Narrative, and Frontend

A second cloud function, deployed apart from the ingestion function, serves the operator view, which lets reads and writes

Beehive & Apiary Monitor | Fog-to-Cloud Deployment Topology

AWS Cloud (us-east-1)

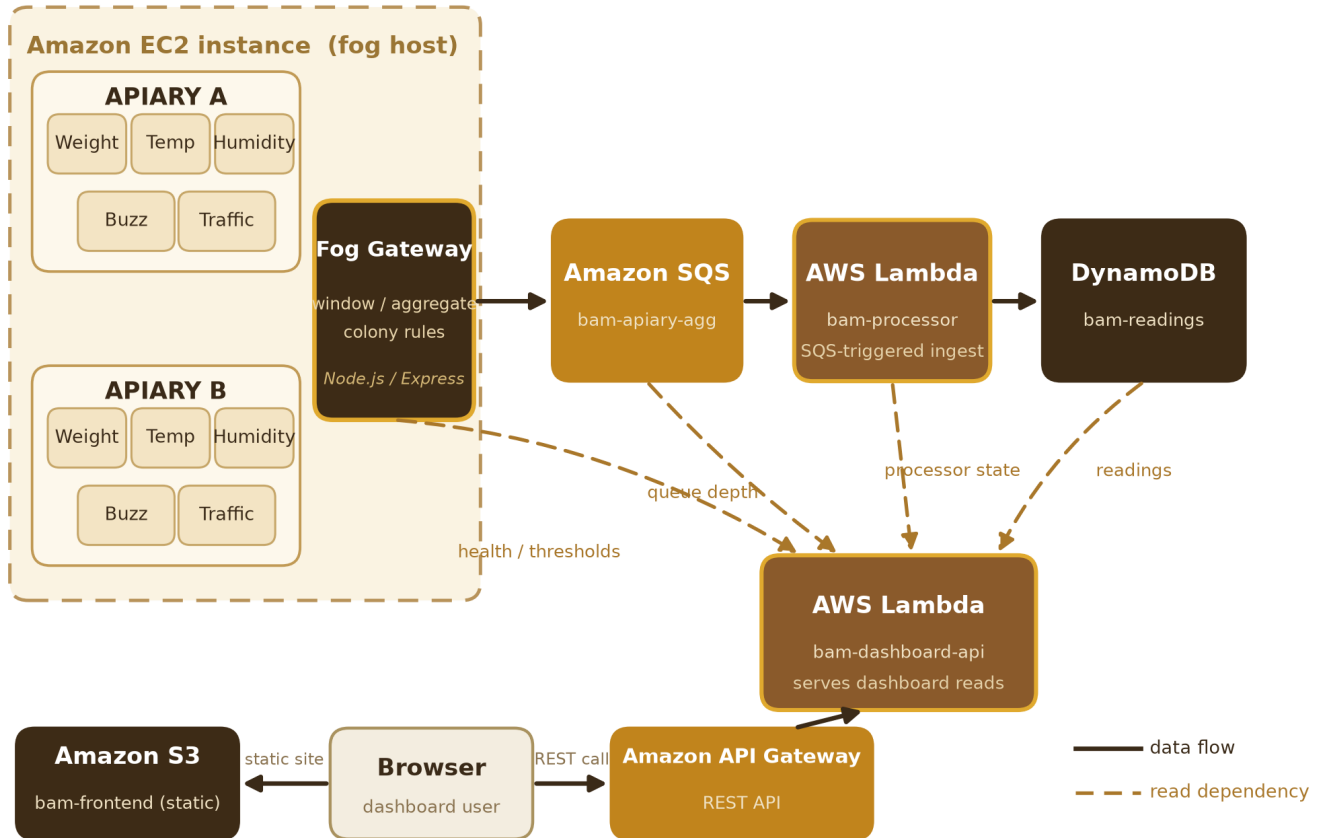


Fig. 1. The deployed fog-to-cloud topology. One edge host at the apiary hosts the fog gateway together with ten sensor containers. Ingestion follows the solid path, hives to queue to ingestion function to table. The dashed paths mark what the dashboard function reads back, the gateway, the queue, the ingestion function, and the table. The browser pulls static assets from object storage and reaches the dashboard interface through the managed gateway.

grow and fail on their own. It offers a small read-only interface: an apiaries endpoint that returns, per apiary, the newest reading on each signal alongside a synthesised colony-health summary; a per-apiary detail endpoint; a readings endpoint that returns a recent series for one signal at one apiary for the trend charts; a health endpoint; a back-end statistics endpoint; and a thresholds endpoint that relays the gateway's published rules. Health arrives as four separate flags, whether the gateway responds, whether the queue is reachable, whether the ingestion function is active, and whether the pipeline is current in that the newest stored window is recent, reported with the age of that window.

The distinctive element of this tier is the colony-health narrative. For each apiary the function reads the recent weight and temperature windows and reduces them to two qualitative states: a weight trend, rising, falling, or steady, obtained by comparing the earliest and latest window means against a small threshold, and a brood thermal state, stable, fluctuating, or critical, obtained from the spread of recent temperature

means and from whether any recent window already carries an alert. These states, together with the count of active alerts, are composed into a single plain-language sentence per apiary, so the dashboard can tell the beekeeper that, for instance, a colony's weight is rising while its brood temperature has breached a safe threshold. Turning telemetry into a judgement is exactly the kind of summarisation the edge-to-cloud split makes room for, and it is placed in the read tier because it depends on a short history of stored windows rather than on any single live reading.

A managed interface gateway with a public REST endpoint fronts the dashboard function. The static frontend, one page, one stylesheet, the page script, and a charting library kept in the repository, comes from object storage. On the page each apiary gets a colony-health summary card bearing its narrative sentence and its live per-signal readings, an alert banner calls out any firing condition, and a panel of window-average trend charts sets the two apiaries side by side, with a header strip showing the four pipeline flags. Every two and a half seconds

the script pulls the apiaries view, health, and back-end statistics again and redraws from each reply. It learns the interface origin from a lone placeholder embedded in the page, swapped for the real gateway origin when the assets upload; run locally, where a single process serves both page and interface, the placeholder stays empty and requests default to the same origin.

E. Alternatives Considered

Every choice below rests on a reason.

Why a queue and not a direct call. The gateway might call the storage function outright and synchronously, dropping the queue along with a hop of delay. That was ruled out: a synchronous call binds the edge to how available and how fast the cloud function is, so throttling it, cooling it, or losing it briefly leaves the gateway stalling or shedding data as pressure backs up to the hives. A queue cuts that bond, soaks up spikes, retries on the consumer’s behalf, and lets the gateway treat a summary as delivered the instant it is queued [8]. On a window cadence the added delay is immaterial, whereas the durability and the separation are exactly what a patchy apiary link demands.

Why a key-value store and not a relational database. The access is narrow and settled in advance: append aggregates in time order, then read back a signal’s recent windows. A key-value store made for that blend of heavy writes and key-based reads holds steady performance as data grows and skips the operational cost of a relational engine, the balance struck by the original Dynamo work and carried on by the managed service [9], [10]. Under the chosen key layout the dashboard’s query reduces to one partition lookup.

Why event-driven functions and not a standing server. Both the ingestion and the interface tier are event-driven functions, well matched to work that comes in bursts and, for the interface, only intermittently: billing is per request rather than for idle time, and the platform grows or shrinks the functions with demand. The acknowledged price is a cold start once a function has idled [11], but its bite is small here, since steady queue traffic keeps the ingestion function warm and frequent dashboard polling does likewise for the interface.

F. Security Posture

Just one inbound port is open on the edge host, and it serves only the gateway’s health and threshold endpoints; with no login key pair and administration handled over the provider’s systems-management channel, the usual remote-access foothold is gone. The two functions share the account’s laboratory role because the account may not create new ones; the production plan is to divide it into least-privilege roles, letting the ingestion function only receive and delete queue messages and write records, and the dashboard function only read. The dashboard interface is left open on purpose, exposing nothing but aggregated, non-personal hive telemetry; and because its page and its interface sit on different origins, every reply, errors among them, carries an explicit cross-origin header so the browser will honour a response returned across origins [12].

TABLE II
AUTOMATED TEST COVERAGE BY MODULE

Module	Tests
Sensors	14
Fog gateway	64
Ingestion function	12
Dashboard interface	48
Total	138

III. IMPLEMENTATION

The whole stack is written in Node.js, with neither a build step nor TypeScript. Express provides the fog gateway’s web layer, and the clients for the queue, the store, and the functions come from the provider’s first-party JavaScript libraries. The dashboard’s trend charts use a charting library committed to the project and served from it, not fetched from a content delivery network, so nothing third-party loads at runtime. In development and in continuous integration the entire stack runs under Docker Compose over a local emulator of the cloud services, and switching between that emulator and the live account is a matter only of the endpoint and credentials supplied to otherwise identical code.

A. Automated Test Suite

Testing relies on the runner that ships with the Node.js runtime, with no outside framework. Since cloud-facing code takes its client as a parameter, the unit tests supply small handwritten fakes rather than a live service or the emulator, so the suite stays quick, repeatable, and off the network. One test deliberately pins down a serverless-runtime quirk: because that runtime hands a handler a completion callback as its third argument, the dashboard handler accepts an injected-client argument only when it genuinely carries a store client, keeping the runtime’s callback from ever being taken for injected clients. In total the suite holds 138 tests over the four modules, split as in Table II. The fog module owns the biggest share, being where the windowing, the statistics, and the colony rules sit.

B. Continuous Integration

On every push and pull request a continuous-integration workflow fires two jobs, the second dependent on the first. Job one installs dependencies and runs each of the four Node.js modules’ unit suites. Job two runs only when job one has passed: it raises the emulator-backed Docker Compose stack, drives an end-to-end check that sends data through the sensors, the gateway, the queue, the ingestion function, and the store, and then brings the stack down. Holding the integration job behind the unit job keeps everyday feedback quick, as a logic mistake trips the fast unit suite, while still exercising the whole assembled system before any change is let in.

C. Infrastructure-as-Code Deployment

The system went live on a real provider laboratory account in the North Virginia region. Provisioning is entirely infrastructure as code, run inside an isolated workspace so the plan sees only this project's resources: one apply stood up twenty-four resources with no commands run by hand, the plan listing twenty-four to add and nothing to change or destroy. Those resources are a key-value table, a message queue, the ingestion and dashboard functions behind a REST interface gateway, an edge compute instance whose firewall permits only the single gateway port and which holds no login key pair, a fixed public address tied to that instance, and two object-storage buckets, one for the dashboard frontend and one for staging the deployment. Both the application code and the infrastructure-as-code configuration live in the repository at [GitHub repository URL].

The system was then evaluated by measuring the running deployment, not by inspecting code alone; all figures below were read from the live endpoint.

IV. LIVE EVALUATION

A. End-to-End Liveness

About a minute after the edge instance had finished starting its containers, the health endpoint showed every one of the four indicators true, the gateway reachable, the queue reachable, the ingestion function active, and the pipeline current, with the newest stored window only seconds old. An age in single digits tells us a reading taken at a hive had already run through the sensor, the gateway, the queue, the ingestion function, and the store and was coming back on the dashboard, the whole trip taking seconds, which is what marks a truly connected pipeline off from a set of components that merely sit deployed side by side.

B. Accumulation and Colony Narratives

Polled again and again through the check, the stored record count kept rising poll on poll, which confirms the ingestion function was steadily draining the queue and writing rather than doing so once. Across that same span the apiaries endpoint served live data for both apiaries, five current readings each with their window range and mean and, most importantly, a synthesised colony-health sentence for each. During verification both apiaries' brood-nest temperature sat in the overheat band, and the pipeline behaved end to end as designed: the edge rule raised a brood-overheat alert on each window, the alerts propagated through the queue and the store to the read tier, each apiary's narrative reported that its brood temperature had breached a safe threshold while its weight was rising, and the dashboard's alert banner named both firing conditions. All static frontend assets were fetched successfully from object storage, the cross-origin header was present on the interface responses, and the placeholder interface origin in the page had been swapped for the real gateway address at upload time.

Fig. 2 is a screenshot of the deployed dashboard rendered in a real browser. It shows the two colony-health summary cards with their narrative sentences and live readings, the alert

banner naming the two brood-overheat conditions, the panel of window-average trend charts, and the header indicators reporting the pipeline healthy. The screenshot shows the system delivering correct, live, interpreted content to a person rather than only raw numbers.

C. Cost

By design the running cost stays low. Every service in the pipeline but the edge instance charges per request and, at the volumes here, remains within the provider's free tier, the queue, the two functions, the key-value store, the interface gateway, and the object storage each billing only when used. The edge instance alone bills continuously, and because the serverless dashboard and interface do not rely on it, only fresh readings and the gateway's own health check do, it can be paused while idle without dropping the dashboard. Idle, then, the system amounts to the cost of one small instance, and even that can be stopped.

D. Limitations

Three limitations deserve stating outright. First, the sensor and fog tier lives on a single edge instance and so is that tier's single point of failure: halt the instance and fresh readings cease, though the already-stored data and the serverless read path stay reachable. A production apiary network would replicate the fog tier or set a gateway at each site. Second, since the laboratory account is session based, only single-session behaviour was confirmed; stability over several days and credential rotation across long runs went unexercised. Third, the two functions currently share one broadly scoped account role; the intended split into least-privilege roles could not be carried out in an account forbidden from creating roles.

V. CONCLUSION

The intent of this work was to take apiary monitoring away from rare, disruptive inspections toward observation that is continuous, non-invasive, and interpreted, built on a distributed design that assigns each task its proper place: windowing, statistical reduction, and colony-rule checks on a fog tier at the apiary, durable storage and elastic serving in the cloud, and a synthesised health judgement at the read tier. What resulted is a working pipeline over two apiaries and five signals each, raising brood overheating, chilling, starvation, and the acoustic precursor of swarming at the edge, and presenting both live readings and a plain-language colony-health narrative from a real cloud deployment.

A few lessons stand out. The message queue proved its worth through resilience rather than latency: putting distance between a patchy apiary uplink and the cloud consumer is what lets the pipeline weather the conditions remote monitoring has to endure. The sort key showed how much a small modelling choice can carry, since attaching the apiary label to a timestamp is the whole difference between two apiaries coexisting and one quietly overwriting the other. The most rewarding lesson was about where interpretation belongs: telemetry becomes useful to a beekeeper only once it is turned

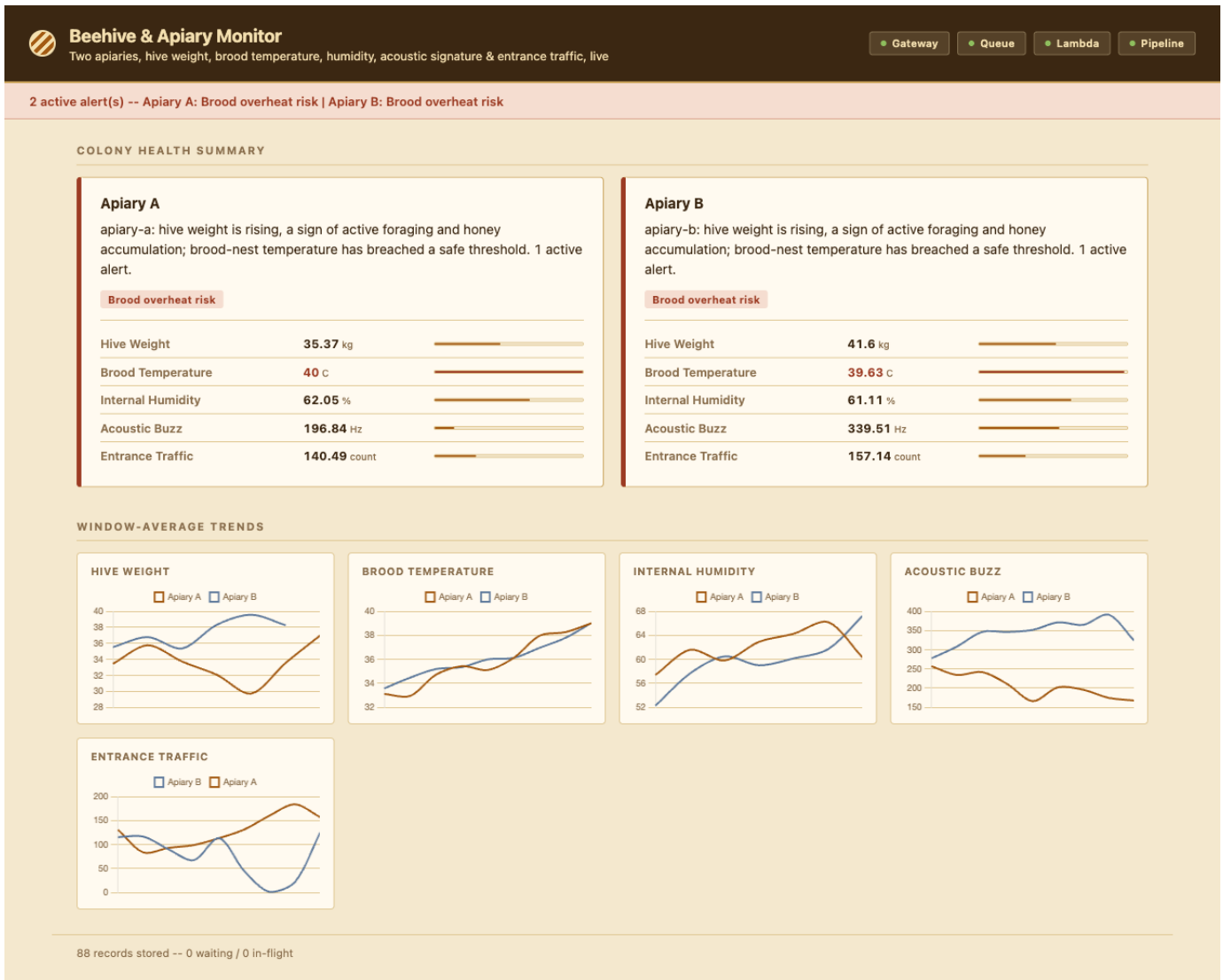


Fig. 2. The deployed operator dashboard rendered live in a browser. Each apiary card carries a plain-language colony-health sentence and its five live readings against their plausible ranges; the alert banner names the two firing brood-overheat conditions; the lower panel compares window-average trends across the two apiaries; the header carries the four pipeline health indicators.

into a judgement, and the natural place for that synthesis is the read tier, where a short history of stored windows already sits ready. Live verification bore this out end to end, when a real brood-overheat excursion made its way from an edge rule all the way to a named condition in the operator's narrative. Sensible next steps are to replicate the fog tier for availability, split the shared role into least-privilege roles, and carry the edge rules from fixed thresholds toward trend-based detection, so that a slow fall in stores or a gradual shift in the acoustic signature is caught from its trajectory rather than only when it crosses a line.

REFERENCES

- [1] A. Zacepins, V. Brusbardis, J. Meitalovs, and E. Stalidzans, "Challenges in the development of precision beekeeping," *Biosystems Engineering*, vol. 130, pp. 60–71, 2015.
- [2] W. G. Meikle and N. Holst, "Application of continuous monitoring of honeybee colonies," *Apidologie*, vol. 46, no. 1, pp. 10–22, 2015.
- [3] I. F. Akyildiz, W. Su, Y. Sankarasubramaniam, and E. Cayirci, "Wireless sensor networks: A survey," *Computer Networks*, vol. 38, no. 4, pp. 393–422, 2002.
- [4] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [5] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the internet of things," in *Proc. 1st Edition of the MCC Workshop on Mobile Cloud Computing (MCC '12)*, 2012, pp. 13–16.
- [6] R. K. Naha, S. Garg, D. Georgakopoulos, P. P. Jayaraman, L. Gao, Y. Xiang, and R. Ranjan, "Fog computing: Survey of trends, architectures, requirements, and research directions," *IEEE Access*, vol. 6, pp. 47 980–48 009, 2018.
- [7] P. Mell and T. Grance, "The NIST definition of cloud computing," National Institute of Standards and Technology, Tech. Rep. Special Publication 800-145, 2011.
- [8] G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003.
- [9] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman,

- A. Pilchin, S. Sivasubramanian, P. Voshall, and W. Vogels, "Dynamo: Amazon's highly available key-value store," in *Proc. 21st ACM SIGOPS Symp. Operating Systems Principles (SOSP '07)*, 2007, pp. 205–220.
- [10] M. Elhemali, N. Gallagher, N. Gordon, J. Idziorek, R. Krog, C. Lazier, E. Mo, A. Mritunjai, S. Perianayagam, T. Rath, S. Sivasubramanian, J. C. Sorenson, S. Sosothikul, D. Terry, and A. Vig, "Amazon DynamoDB: A scalable, predictably performant, and fully managed NoSQL database service," in *Proc. USENIX Annual Technical Conf. (USENIX ATC '22)*, 2022, pp. 1037–1048.
- [11] L. Wang, M. Li, Y. Zhang, T. Ristenpart, and M. Swift, "Peeking behind the curtains of serverless platforms," in *Proc. USENIX Annual Technical Conf. (USENIX ATC '18)*, 2018, pp. 133–146.
- [12] A. Barth, "The web origin concept," Internet Engineering Task Force, Tech. Rep. RFC 6454, 2011.